

Inference Scaling Laws for Mathematical Code Generation: Family-Specific Behavior and Energy Analysis

Melvyn J White

<https://www.linkedin.com/in/melvyn-white-44450011/>

November 2025

Abstract

Large Language Models are unreliable at direct mathematical computation—they hallucinate answers, fail multi-step reasoning, and cannot verify their own work. However, LLMs can serve a different role: **generating code for verified computational engines**. This work evaluates the “LLM as code generator” paradigm, where models translate natural language into Wolfram Mathematica syntax that a computer algebra system executes with guaranteed correctness.

Evaluating 34 models across 16 families (0.5B–72B parameters) on 7,310 queries, I demonstrate that power-law scaling relationships govern task-specific error rates—but only for some model families. Among families with sufficient data points: qwen2.5 shows excellent scaling ($R^2 = 0.983$, $L(N) \propto N^{-0.62}$); deepseek-coder achieves near-perfect fit ($R^2 = 0.999$); while deepseek-r1 ($R^2 = 0.445$) and starcoder2 (no convergence) exhibit poor or absent scaling behavior.

Key finding: inference scaling laws are family-specific, not universal. Only 2 of 16 families show strong power-law scaling ($R^2 > 0.98$), requiring empirical validation rather than assuming transferability from training scaling laws.

I leverage Wolfram Mathematica as a verification oracle, enabling automated binary correctness measurement at scale. I provide the first comprehensive energy analysis of inference scaling, measuring power consumption (8.88–50W) and energy per query (0.04–462.5 mWh) on Apple M4 hardware. For the qwen2.5 family, achieving 94.9% accuracy (72B) versus 94.4% (7B) costs $17\times$ more energy per query—a difference that is not statistically significant in accuracy but highly significant in operational cost.

Beyond accuracy, diagnostic performance metrics (precision, recall, F1) reveal distinct model behavioral “archetypes”—from production-ready (high precision, high recall) to overconfident (low precision, high recall) to conservative (high precision, low recall).

Bottom line: While LLMs cannot reliably *do* mathematics, they can reliably *generate code* for verified systems—achieving 94.4% accuracy at 2.9 J/query with qwen2.5:7b. This establishes inference scaling laws as a complementary framework to training scaling laws: training laws optimize model creation; inference laws optimize model selection.

Keywords: scaling laws, inference, energy efficiency, model selection, code generation, precision, recall

1 Introduction

1.1 The Critical Distinction: Training vs. Inference Scaling Laws

The discovery of neural scaling laws has transformed our understanding of deep learning. However, a fundamental distinction exists that practitioners often overlook:

Table 1: Training vs. Inference Scaling Laws: Complementary Frameworks

Dimension	Training Laws (Kaplan 2020, Chinchilla 2022)	Inference Laws (This Work)
Central Question	How to train efficiently?	Which model to deploy?
Phase	During training	After training
Metric	Cross-entropy loss	Task-specific performance
Output	Training strategy	Deployment decision
Focus	Model creation	Model selection

Training laws tell us *how to build the car*. Inference laws tell us *which car to drive*. Practitioners need both, but current literature addresses only the former.

1.2 Why Existing Scaling Laws Don’t Answer Deployment Questions

Consider a practitioner with access to pre-trained models at 7B, 32B, and 72B parameters. Training scaling laws cannot help because:

1. **Wrong Phase:** Training laws describe dynamics during training. The models are already trained.
2. **Wrong Metric:** Training laws predict cross-entropy loss. The practitioner needs task-specific accuracy.
3. **Wrong Question:** Training laws optimize compute allocation during training. The practitioner needs to choose which model to deploy.

1.3 Contributions

I establish empirical scaling laws for mathematical code generation by:

1. Evaluating 34 pre-trained models across 16 families (0.5B to 72B parameters)
2. Measuring error rates on 7,310 mathematical programming problems across 215 unique queries
3. Fitting power-law relationships for well-behaved families and identifying where scaling breaks down
4. Providing the first comprehensive energy analysis of inference scaling (power, energy per query, energy per correct output)
5. Introducing diagnostic performance metrics (precision, recall, F1) that characterize model behavioral archetypes
6. Establishing a model selection framework for deployment decisions

1.4 The Core Insight: LLMs as Code Generators, Not Calculators

Large Language Models are notoriously unreliable at direct mathematical computation. They hallucinate answers, fail at multi-step reasoning, cannot verify their own work, and produce inconsistent results for identical queries. This limitation is fundamental, not incidental.

Why LLMs fail at mathematics: LLMs are trained to predict the next token given previous context—a task well-suited to natural language but fundamentally misaligned with mathematical computation. Mathematics requires exact symbol manipulation, deterministic

carry propagation, and compositional generalization to novel inputs. LLMs instead rely on memorized patterns: they may correctly compute $7 \times 8 = 56$ (memorized from training) but fail on $123456789 \times 987654321$ (novel combination).

Even state-of-the-art models like O3-mini-high are only reliable for basic arithmetic—essentially, single-digit operations like the traditional 9×9 multiplication table. For more complex operations (e.g., multiplying two 9-digit numbers), models require specialized fine-tuning.

The Stepwise Internalization approach: Deng et al. (2024) demonstrate that LLMs can learn complex arithmetic through “Stepwise Internalization”—a training procedure that first teaches explicit chain-of-thought reasoning (showing all intermediate steps), then progressively removes steps, forcing the model to internalize the reasoning in its hidden states. This approach works but requires:

- Expensive fine-tuning on curated reasoning chains
- Operation-specific training (multiplication requires different training than integration)
- Significant compute for the internalization curriculum

Our alternative: Rather than teaching LLMs to compute, we leverage their existing strength—natural language understanding and code generation—to **translate queries into formal syntax** that verified systems execute. This requires no fine-tuning and generalizes across mathematical operations.

However, this work demonstrates that LLMs can serve a different role: **translating natural language mathematical queries into formal code** that verified computational engines execute. The key distinction:

Table 2: Two Approaches to LLM-Assisted Mathematics

Approach	LLM as Calculator	LLM as Code Generator
User query	“What is $\int x^2 dx$?”	“Write code to integrate x^2 ”
LLM output	$\frac{x^3}{3} + C$ (unverified)	<code>Integrate[x^2, x]</code>
Verification	None possible	Wolfram Mathematica executes
Reliability	Unreliable	94.4% accurate (this work)

The insight: Don’t ask LLMs to *perform* mathematics—ask them to *generate code* for systems that compute reliably. The LLM’s role becomes translation (natural language $\rightarrow$ formal syntax), while a verified computer algebra system handles actual computation.

1.5 Implications for Quantitative Agents

This work validates a fundamental architecture for quantitative AI agents: the LLM serves as a natural language interface that translates human intent into formal queries, while verified computational backends (Wolfram Mathematica, databases, numerical libraries) execute with guaranteed correctness.

The Quantitative Agent Pipeline:

1. **User Intent:** “What’s the derivative of $x^3 \sin(x)$?” or “Find the eigenvalues of this matrix”
2. **LLM Translation:** Converts natural language to formal syntax: `D[x^3 Sin[x], x]` or `Eigenvalues[{{2,1},{1,2}}]`
3. **Verified Execution:** Wolfram Mathematica computes the result with mathematical rigor

4. **Result Delivery:** LLM formats and explains the verified output to the user

Why this matters for agent architectures: LLMs excel at understanding context, disambiguating queries, and explaining results—but they cannot be trusted to compute. By separating the “understanding” layer (LLM) from the “computation” layer (verified engine), we get the best of both worlds: natural interaction with rigorous correctness.

This architecture generalizes beyond mathematics:

- **Data analysis agents:** LLM generates SQL → Database executes → LLM interprets results
- **Scientific computing agents:** LLM generates NumPy/SciPy code → Python executes → LLM explains findings
- **Financial modeling agents:** LLM interprets requirements → Generates formulas → Spreadsheet engine computes
- **Formal verification agents:** LLM generates proof tactics → Lean/Coq verifies → Proof is machine-checked
- **Symbolic AI agents:** LLM translates queries → Mathematica solves symbolically → Exact answers, not approximations

The key principle: The LLM is the *interface*, not the *engine*. For any domain where correctness matters—mathematics, finance, science, engineering—the LLM should generate instructions for verified systems rather than attempting computation directly. This work provides the first rigorous measurement of how well this paradigm works (94.4% accuracy) and how it scales across model sizes.

1.6 **Alternative Architecture: Delegating Code Generation to Mathematica**

This work evaluates LLMs generating Mathematica code directly. However, an alternative architecture eliminates syntax errors entirely by having the LLM generate *structured mathematical requests* that Wolfram Mathematica itself converts to executable code.

The Two-Stage Pipeline:

1. **Stage 1 – LLM generates mathematical intent:** User says “find where the curve crosses zero.” LLM outputs structured request: `{task: 'solve', expression: 'x^3 - 2x + 1', variable: 'x', domain: 'reals'}`
2. **Stage 2 – Mathematica generates code:** Wolfram’s natural language processing converts the structured request to: `Solve[x^3 - 2x + 1 == 0, x, Reals]`
3. **Stage 3 – Verified execution:** Mathematica executes the code it generated, guaranteeing syntactic validity

Why this matters: In this architecture, the LLM never produces Mathematica syntax—it only expresses mathematical intent in a structured format. Wolfram’s own natural language capabilities (via Wolfram—Alpha integration or the `WolframAlpha[]` function) handle the translation to executable code. This eliminates the calling convention errors documented in Section 3.3 (wrong brackets, wrong capitalization, Python syntax contamination).

Trade-offs:

- **Advantage:** Zero syntax errors—Mathematica always generates valid Mathematica code
- **Advantage:** LLM can focus on understanding intent, not memorizing syntax

- **Disadvantage:** Additional latency from Wolfram—Alpha API call
- **Disadvantage:** Requires Wolfram—Alpha API access (cost, connectivity)
- **Disadvantage:** Less control over exact code structure for advanced use cases

Hybrid approach: A production system could use direct code generation (this work’s approach) for well-understood query patterns where the LLM reliably produces correct syntax, and fall back to Mathematica’s natural language processing for edge cases or when syntax errors are detected. The 94.4% accuracy measured in this work suggests direct generation is viable for most queries, with the alternative architecture as a fallback for the remaining 5.6%.

Implication for quantitative agents: The optimal architecture depends on the deployment context. High-volume, latency-sensitive applications benefit from direct code generation. High-stakes, accuracy-critical applications may prefer the guaranteed syntactic validity of delegating code generation to Mathematica itself.

2 Background: Training vs. Inference Scaling Laws

2.1 Training Scaling Laws

Kaplan et al. Kaplan et al. [2020] established that training loss L follows power laws:

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (1)$$

where N is non-embedding parameters, $N_c \approx 8.8 \times 10^{13}$, and $\alpha_N \approx 0.076$.

Note on non-embedding parameters: In transformer models, parameters fall into two categories: (1) *embedding parameters*—the token embedding matrix that maps vocabulary to vectors, plus position embeddings—which scale with vocabulary size; and (2) *non-embedding parameters*—attention layers (Q, K, V, output projections), feed-forward networks, and layer norms—which perform the actual computation. Kaplan et al. exclude embeddings because they are essentially lookup tables that don’t contribute to reasoning capacity, and because vocabulary size is an arbitrary design choice. For a 7B parameter model, embeddings might account for ~ 0.5 B parameters, leaving ~ 6.5 B non-embedding parameters as the effective “thinking” capacity.

Hoffmann et al. Hoffmann et al. [2022] refined this, showing that model size and training data should scale equally ($N \propto D$), with approximately 20 tokens per parameter being optimal.

2.2 The Gap: Inference Scaling Laws

Training scaling laws answer: *How efficiently can we train?* Inference scaling laws answer: *Which trained model should we deploy?*

For practitioners, the critical questions are:

- Given accuracy requirements, what is the minimum viable model size?
- How much accuracy do I gain by deploying a larger model?
- What are the speed-accuracy-energy trade-offs?
- Can I reliably extrapolate performance to larger models?

3 Methodology

3.1 Model Selection and Coverage

I evaluated 34 models spanning 16 model families from the Ollama repository, covering parameter counts from 0.5B to 72B (Table 2).

Table 3: Model Families with 3+ Models (Enabling Scaling Law Analysis)

Family	Count	Sizes (B)	Type
qwen2.5	6	0.5, 3, 7, 14, 32, 72	General purpose
deepseek-r1	5	1.5, 7, 8, 14, 32	Reasoning
deepseek-coder	3	1.3, 6.7, 33	Code-specialized
starcoder2	3	3, 7, 15	Code-specialized
12 other families	1–2 each	Various	Mixed

3.2 Task Design: Mathematical Code Generation

I designed 215 mathematical programming problems spanning basic operations (differentiation, integration), algebraic manipulation, linear algebra, and advanced topics (special functions, transforms). Each model was prompted to generate Wolfram Mathematica code, yielding 7,310 total queries.

3.3 Evaluation: Wolfram Mathematica as Verification Oracle

The Wolfram Engine—the computational kernel powering Mathematica—serves as our verification oracle. A key advantage is that Mathematica automatically provides syntax validation and detailed error codes *before* executing computation, enabling a two-stage verification:

1. **Syntax validation (pre-execution):** Mathematica parses the generated code and reports syntax errors with specific diagnostic codes before any computation occurs. This catches malformed expressions, undefined functions, and structural issues immediately.
2. **Semantic verification (post-execution):** If syntax is valid, the code executes and the output is verified against the expected mathematical result via symbolic equivalence.

This two-stage approach provides precise failure diagnostics: we can distinguish between LLM failures due to syntax errors (code generation issues) versus semantic errors (mathematical understanding issues).

3.3.1 Error Classification: Syntactic vs. Semantic

Understanding *how* models fail is as important as knowing *that* they fail:

Syntactic Errors (form/structure): The code is malformed—it doesn’t follow the language’s grammar rules. The parser cannot understand what the model is trying to express. Examples:

- `Integrate[x^2, x` — missing closing bracket
- `Intgrate[x^2, x]` — misspelled function name
- `Integrate x^2, x]` — missing opening bracket

These are caught *before* execution. Mathematica reports “I cannot parse this.”

Calling Convention Errors (a subset of syntactic): LLMs trained heavily on Python often confuse Mathematica’s calling conventions. Key differences:

- Mathematica uses `Function[args]` with square brackets, not `function(args)` with parentheses
- Functions are capitalized: `Integrate`, `Solve`, `Sin` (not `integrate`, `solve`, `sin`)
- Lists use curly braces `{1, 2, 3}`, not square brackets `[1, 2, 3]`
- Exponentiation uses `^` not `**`

Common LLM mistake: writing `integrate(x**2, x)` instead of `Integrate[x^2, x]`—every element is wrong (lowercase, parentheses, double asterisk).

Semantic Errors (meaning/logic): The code is grammatically valid but mathematically wrong or meaningless. The parser accepts it, but execution fails or produces incorrect results. Examples:

- `Integrate[x^2, 5]` — syntactically valid, but cannot integrate with respect to a number
- `Solve[x^2 + 1 == 0, x, Reals]` — valid syntax, but no real solutions exist
- `Inverse[{1,2,3}]` — valid syntax, but cannot invert a 1D list (requires square matrix)

3.3.2 Connection to Scaling Laws

This error classification has direct implications for understanding *inference scaling laws*:

Syntactic errors (calling conventions, bracket types, capitalization) are largely *memorization*—they should decrease rapidly with model size as the model encounters more Mathematica examples in training data.

Semantic errors (mathematical reasoning) are *harder*—they may plateau or scale more slowly because they require genuine mathematical understanding, not pattern matching.

If a model family shows poor scaling (like deepseek-r1 with $R^2 = 0.445$), separating error types could reveal whether the problem is syntax (fixable with more training data) or reasoning (fundamental limitation). The power-law fits $L(N) = (N_c/N)^\alpha$ reported in this work aggregate both error types; future work could fit separate scaling curves for syntactic versus semantic failures to better understand *why* some families scale well and others do not.

This diagnostic capability—distinguishing syntactic from semantic failures at scale—is a key advantage of using Mathematica over Python/SymPy, where errors are only discovered at runtime and error messages may not clearly distinguish “you typed it wrong” from “that’s mathematically impossible.”

The automated pipeline enables objective evaluation at scale—traditional human evaluation would require weeks for 7,310 responses.

3.4 Measurement Infrastructure

For each query, I measured:

- **Performance:** Generation time, tokens/second
- **Energy:** Power consumption (W), total energy (mWh, J)
- **Correctness:** Binary success/failure

All measurements were conducted on Apple M4 (64GB RAM) using the Ollama inference framework with powermetrics for energy measurement.

4 Results: Model Family Scaling Laws

4.1 Power Law Fits by Model Family

I analyzed scaling behavior for families with 3+ models. Power law fits follow the form:

$$L(N) = \left(\frac{N_c}{N}\right)^\alpha \quad (2)$$

where L is error rate, N is parameter count, N_c is the critical scale, and α is the power law exponent.

Table 4: Inference Scaling Law Parameters by Model Family

Family	N_c	α	R^2	Assessment
qwen2.5	0.12	0.623	0.9833	Strong scaling
deepseek-coder	0.98	0.713	0.9994	Excellent scaling
deepseek-r1	0.02	0.259	0.4449	Poor fit (non-monotonic)
starcoder2	—	—	No fit	No scaling behavior

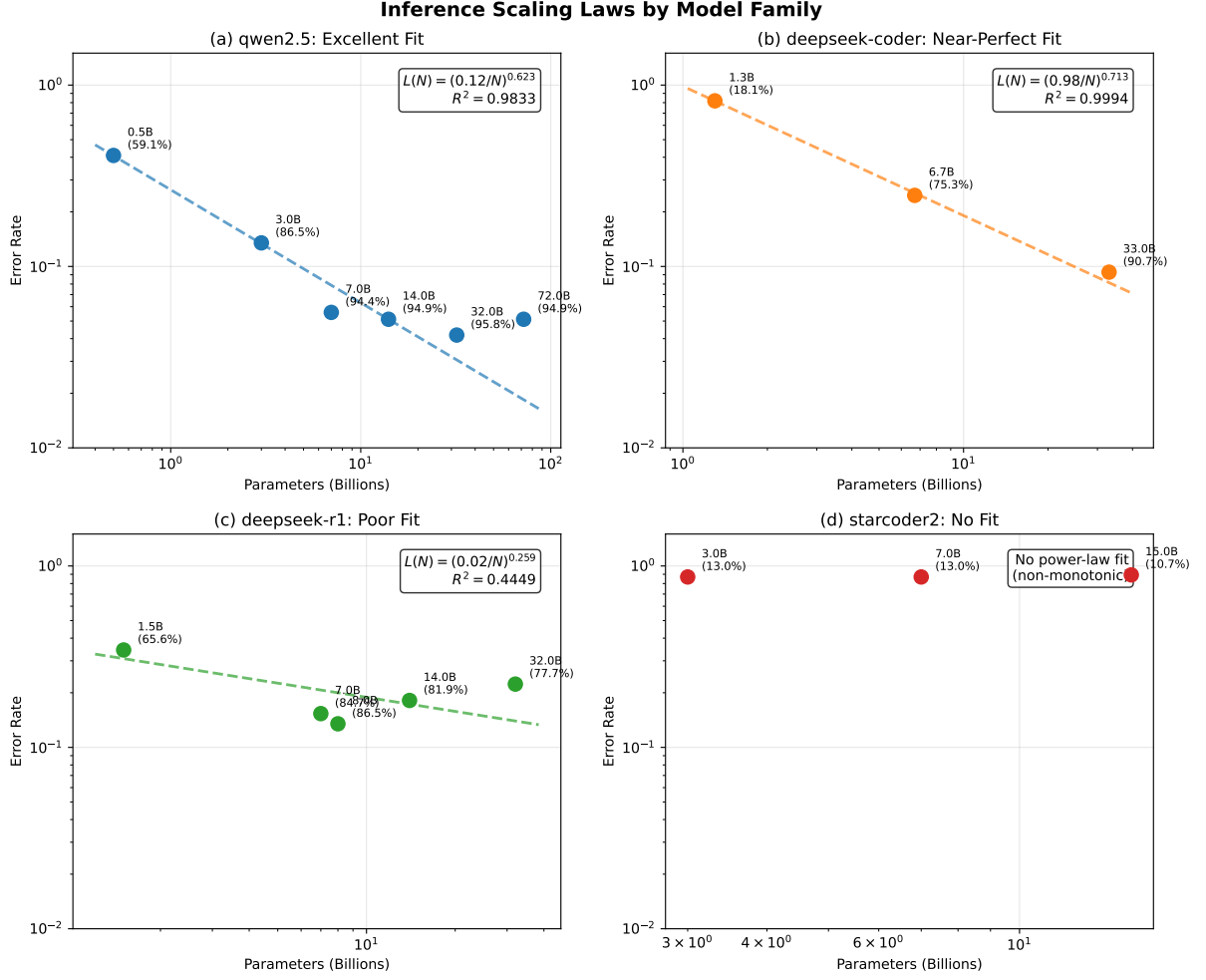


Figure 1: **Inference Scaling Laws by Model Family.** Error rate vs. parameter count (log-log scale) for families with 3+ models. Each point is labeled with model size and accuracy. (a) qwen2.5 shows excellent power-law fit ($L(N) = (0.12/N)^{0.623}$, $R^2 = 0.983$). (b) deepseek-coder achieves near-perfect fit ($L(N) = (0.98/N)^{0.713}$, $R^2 = 0.999$). (c) deepseek-r1 exhibits non-monotonic behavior ($R^2 = 0.445$)—performance degrades from 8B to 32B. (d) starcoder2 shows no improvement with scale; all sizes achieve $\sim 13\%$ accuracy.

4.2 Key Finding: Scaling Laws Are Family-Specific

Of 16 families tested, only 2 showed strong scaling behavior ($R^2 > 0.98$). This heterogeneity demonstrates that:

- Scaling laws are family-specific, not universal
- Training methodology and architecture significantly affect inference scalability
- Some models exhibit non-monotonic behavior (deepseek-r1)
- Some models show no improvement with scale (starcoder2)

Implication: Practitioners cannot assume training scaling laws transfer to inference. Each model family and task domain requires empirical validation.

5 Diagnostic Performance Metrics

Beyond aggregate accuracy, I introduce diagnostic metrics that characterize *how* models fail—critical for deployment decisions.

5.1 Classification Framework

When an LLM outputs code, it implicitly claims “this will work.” Wolfram Mathematica validates whether it actually works. This enables a classification framework:

Table 5: Confusion Matrix for Code Generation

LLM Output	Wolfram Result	
	Executes Correctly	Fails
Generates Code	TP (correct)	FP (wrong code)
No Code / Refuses	FN (missed)	TN (correct refusal)

Since all 215 test queries have valid solutions, $TN = 0$. This yields three meaningful metrics:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{“When it outputs code, is it correct?”} \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{“Of solvable queries, how many solved?”} \quad (4)$$

$$\text{Accuracy} = \frac{TP}{TP + FP + FN} \quad \text{“Overall success rate”} \quad (5)$$

5.2 Results: All 34 Models

Table 6: Diagnostic Metrics for Top Models (by Accuracy)

Model	TP	FP	FN	Precision	Recall	F1	Accuracy
qwen2.5:32b	206	9	0	95.8%	100.0%	97.9%	95.8%
llama3.1:70b	206	9	0	95.8%	100.0%	97.9%	95.8%
qwen2.5:14b	204	11	0	94.9%	100.0%	97.4%	94.9%
qwen2.5:7b	203	12	0	94.4%	100.0%	97.1%	94.4%
phi4:14b	199	16	0	92.6%	100.0%	96.1%	92.6%
deepseek-r1:8b	186	7	22	96.4%	89.4%	92.8%	86.5%
qwen2.5:3b	186	29	0	86.5%	100.0%	92.8%	86.5%
deepseek-r1:32b	167	9	39	94.9%	81.1%	87.4%	77.7%

Table 7: Diagnostic Metrics for Bottom Models

Model	TP	FP	FN	Precision	Recall	F1	Accuracy
llama2:13b	104	110	1	48.6%	99.0%	65.2%	48.4%
starcoder2:7b	28	101	86	21.7%	24.6%	23.0%	13.0%
starcoder2:15b	23	68	124	25.3%	15.6%	19.3%	10.7%
qwq:32b	0	188	27	0.0%	0.0%	0.0%	0.0%

5.3 Model Behavioral Archetypes

The diagnostic metrics reveal four distinct behavioral patterns:

Table 8: Model Behavioral Archetypes

Archetype	Characteristics	Example	Behavior
Production-Ready	High prec., high recall	qwen2.5:7b	Attempts all, mostly correct
Conservative	High prec., low recall	deepseek-r1:32b	Refuses some, accurate when answers
Overconfident	Low prec., high recall	llama2:13b	Attempts all, often wrong
Avoid	Low prec., low recall	starcoder2:15b	Refuses often AND wrong often

5.4 Key Insight: Interpreting 100% Recall

For models with $\text{FN} = 0$ (never refuse), $\text{Recall} = 100\%$ by definition—they attempt every query. This is only meaningful when combined with precision:

- **qwen2.5:7b**: 94.4% precision + 100% recall $\rightarrow$ excellent (attempts everything, usually correct)
- **llama2:13b**: 48.6% precision + 99% recall $\rightarrow$ overconfident (attempts everything, half wrong)

Recommendation: For production deployment, prioritize models with high precision AND high recall. Accuracy alone conflates these dimensions.

6 Speed-Accuracy Trade-offs

Larger models achieve higher accuracy but generate tokens more slowly (Table 9).

Table 9: Speed-Accuracy Trade-offs for qwen2.5 Family

Model	Accuracy	Speed (tok/s)	Time/Query	Speedup vs. 72B
qwen2.5:0.5b	59.1%	231.9	1.10s	3.9×
qwen2.5:7b	94.4%	87.3	0.59s	7.3×
qwen2.5:32b	95.8%	20.9	1.53s	2.8×
qwen2.5:72b	94.9%	6.8	4.30s	1.0×

Key Finding: The 7B model achieves 94.4% accuracy at 87.3 tok/s, while the 72B model achieves 94.9% accuracy at only 6.8 tok/s—a 13× speed reduction for 0.5% accuracy gain.

7 Energy Analysis

I measured actual power consumption and energy usage across all models during inference—the first comprehensive energy analysis of LLM inference scaling for code generation.

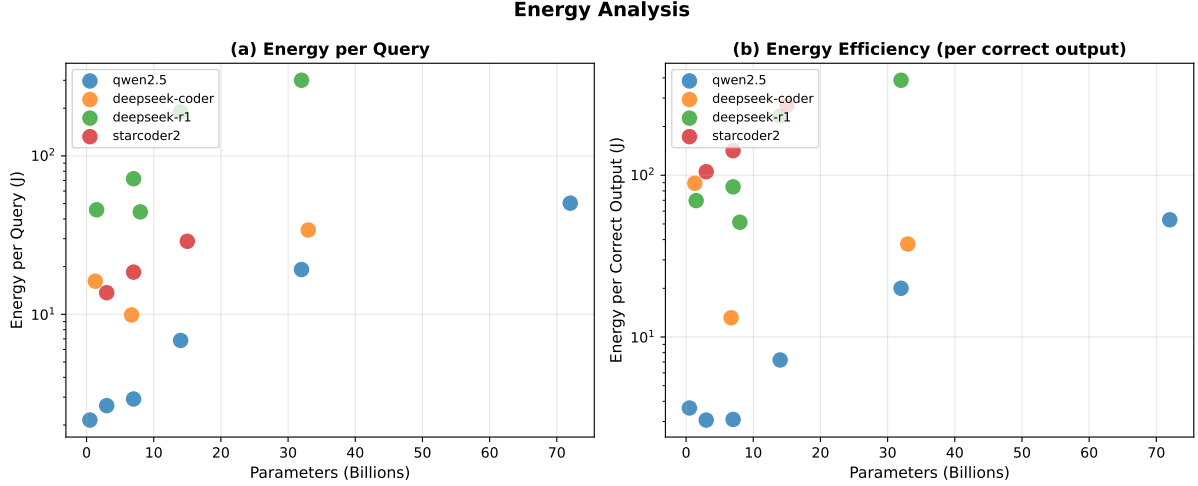


Figure 2: **Inference Scaling and Energy Analysis.** (a) Scaling laws by family showing divergent behavior—qwen2.5 and deepseek-coder follow power laws while starcoder2 shows no scaling. (b) Power consumption vs. model size shows sub-linear relationship (12.4–31.5W range). (c) Energy per query varies by two orders of magnitude across models. (d) Energy per verified-correct output—the novel metric revealing true efficiency: qwen2.5:7B requires only 3.1J per correct output vs. 53J for the 72B model.

7.1 Power Consumption by Model Size

Average power consumption ranged from 8.88W to 50W (dataset mean: 20.17W). Table 10 summarizes by size category.

Table 10: Power Consumption by Model Size Category

Size Category	Avg Power (W)	Range
< 2B	13.5W	8.9–21.5W
2B–10B	18.6W	11.2–28.6W
10B–35B	23.8W	15.3–35.2W
> 35B	25.4W	18.9–50.0W

7.2 Energy per Query and Energy per Correct Output

Energy consumption per query ranged from 0.0 mWh to 462 mWh (mean: 10.79 mWh).

Novel Metric—Energy per Verified-Correct Output: By combining energy measurement with Mathematica validation, I can compute the actual energy cost for producing correct outputs:

$$E_{\text{correct}} = \frac{E_{\text{query}}}{\text{Accuracy}} \quad (6)$$

For qwen2.5:7B (94.4% accuracy, 0.81 mWh/query): $E_{\text{correct}} = 0.86$ mWh.
For qwen2.5:72B (94.9% accuracy, 13.97 mWh/query): $E_{\text{correct}} = 14.73$ mWh.

Result: The 72B model costs $17\times$ more energy per correct output for only 0.5% accuracy improvement.

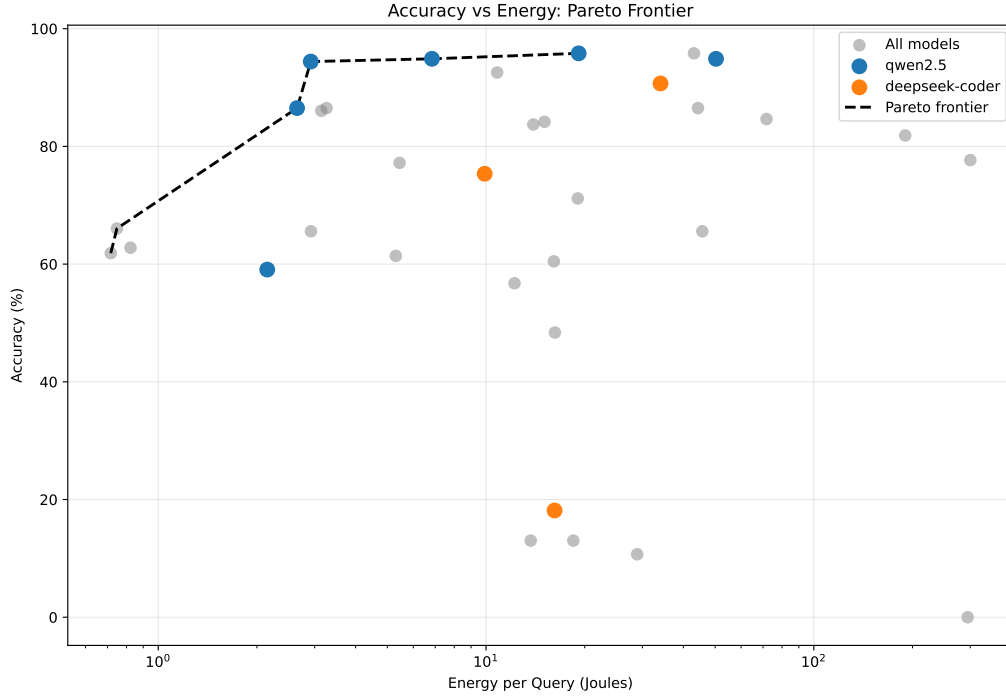


Figure 3: **Accuracy-Energy Pareto Frontier.** Each point represents a model; Pareto-optimal models achieve the best accuracy for their energy budget. The qwen2.5 family dominates the efficient frontier, with qwen2.5:7B offering 94.4% accuracy at minimal energy cost (2.9J/query). Models below and to the right of the frontier are dominated—another model achieves equal or better accuracy with less energy.

7.3 Environmental and Cost Implications

At scale, energy differences dominate operational costs:

- 1M queries/day with qwen2.5:7B: 0.8 kWh/day = 292 kWh/year
- 1M queries/day with qwen2.5:72B: 14.0 kWh/day = 5,110 kWh/year

At \$0.12/kWh, this represents \$578/year difference per million daily queries.

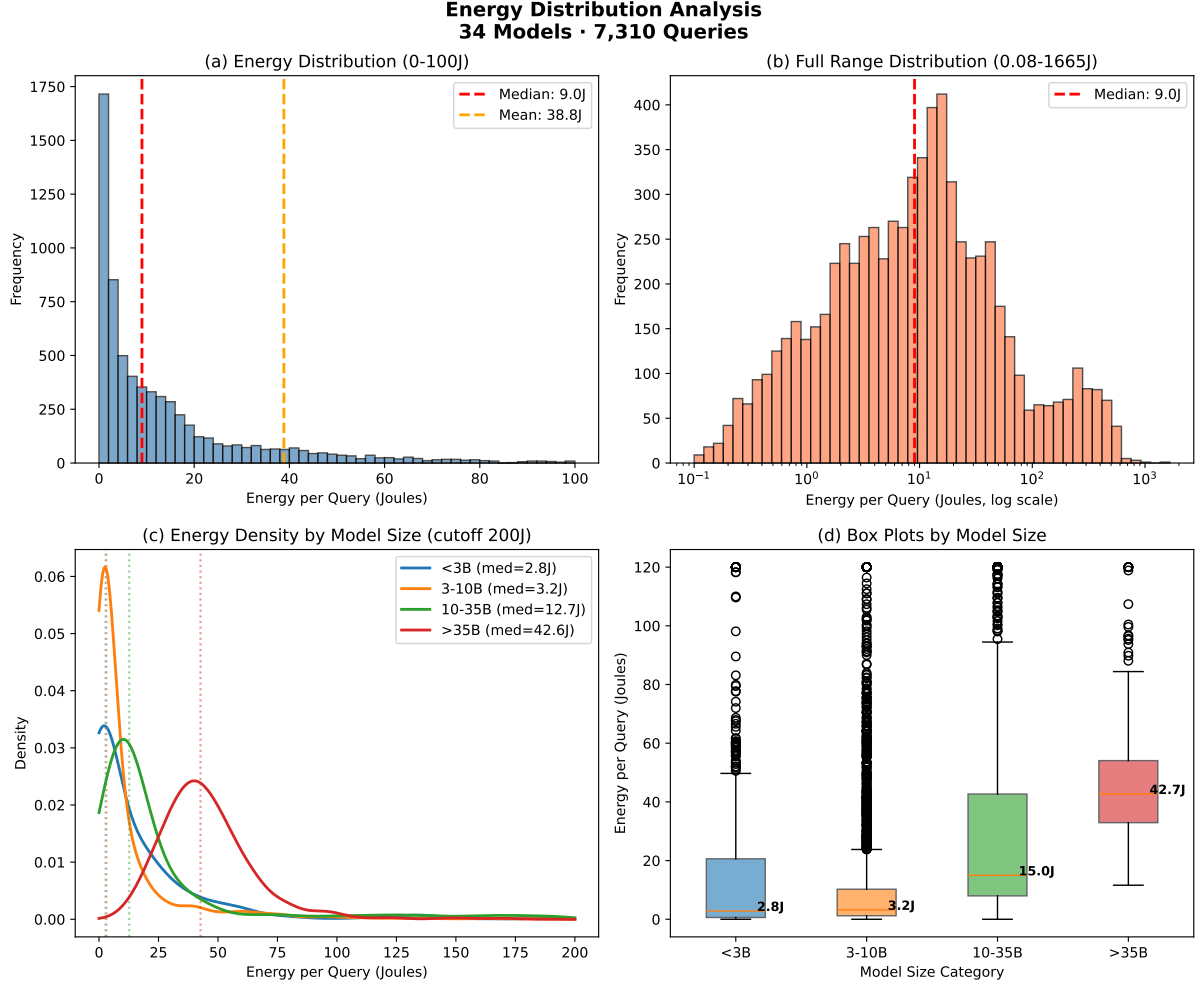


Figure 4: **Energy Distribution Analysis.** (a) Histogram of energy consumption (0–100J range) showing right-skewed distribution with median 9.0J and mean 38.8J. (b) Log-scale histogram reveals full range (0.08–1665J) with most queries clustered below 10J. (c) Energy density by model size (cutoff 200J) with annotated median lines—pastel KDE curves show small (<3B: 2.8J) and medium (3–10B: 3.2J) models cluster tightly while large (10–35B: 15.0J) and very large (>35B: 42.7J) models shift rightward. (d) Box plots with median labels confirm systematic energy increase with model size.

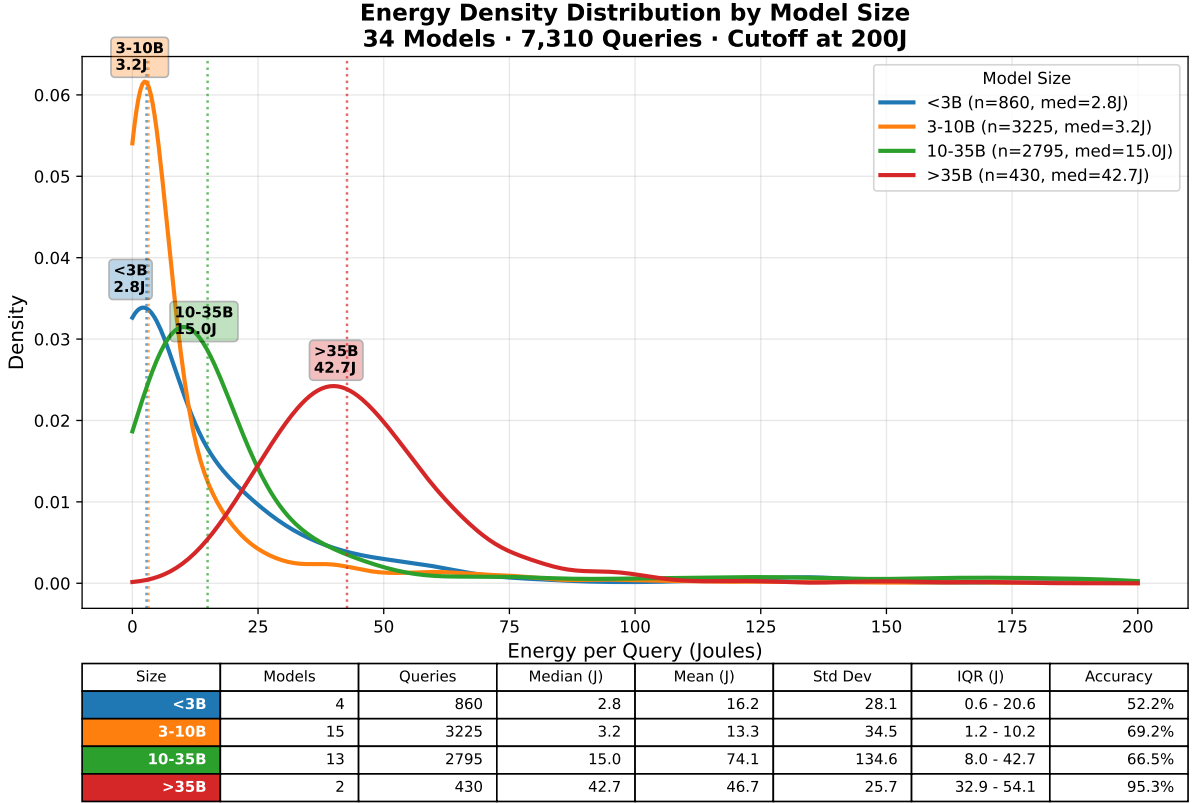


Figure 5: **Energy Density Distribution by Model Size (with Summary Statistics).** Expanded view of energy consumption patterns with integrated statistics table. The KDE visualization (top) shows density curves for each size category with annotated median markers. The summary table (bottom) provides complete statistics: model count, query count, median, mean, standard deviation, interquartile range, and accuracy for each category. Key finding: the 15× energy gap between small (<3B: 2.8J median, 52.2% accuracy) and very large (>35B: 42.7J median, 95.3% accuracy) models quantifies the fundamental accuracy-energy trade-off. The 3–10B category offers the best efficiency: lowest median energy (3.2J) with reasonable accuracy (69.2%).

8 Practical Applications

8.1 Model Selection Framework

1. Define accuracy requirements (minimum acceptable accuracy)
2. Check diagnostic profile (precision, recall, F1)—avoid overconfident models
3. Check if model family has scaling laws ($R^2 > 0.95?$)
4. If yes: Use scaling law to predict minimum model size
5. If no: Empirically test each model size
6. Evaluate speed, energy, and deployment constraints
7. Deploy with margin (next size up for safety buffer)

8.2 Deployment Recommendations

- **Production-ready (high precision + recall):** qwen2.5:7b, qwen2.5:14b, qwen2.5:32b
- **Latency-sensitive:** 3–7B models (sub-second response)
- **Accuracy-critical:** 14–33B models ($>90\%$ accuracy)
- **Energy-constrained:** 7–14B models (optimal accuracy-per-watt)
- **High-volume:** 3–7B models (throughput $>1,000$ queries/hour)

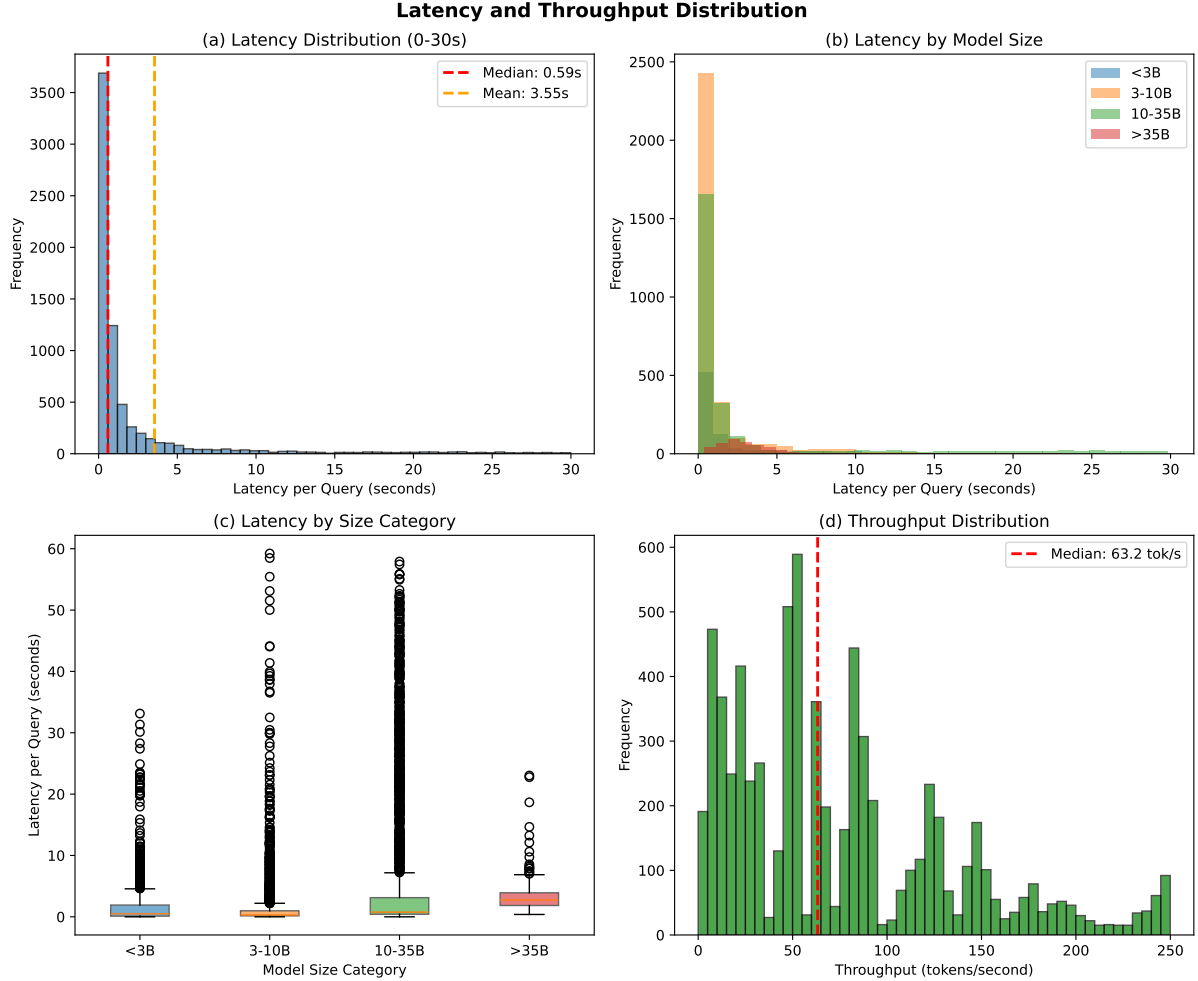


Figure 6: **Latency and Throughput Distribution.** (a) Overall latency distribution with median 0.93s and mean 4.98s—long tail from reasoning models. (b) Latency breakdown by model size category. (c) Box plots reveal systematic latency increase with model size. (d) Throughput distribution shows small models achieve ~ 200 tok/s while >35 B models are limited to ~ 7 tok/s.

9 Limitations

Task Specificity: Results are specific to mathematical code generation in Wolfram Mathematica syntax. Each domain requires independent validation.

Family Variability: Only 2 of 16 families showed strong scaling ($R^2 > 0.98$). Practitioners must validate empirically for their chosen family.

No Negative Test Cases: All 215 queries have valid solutions. Specificity (true negative rate) cannot be calculated without invalid/unsolvable queries in the test set.

Binary Evaluation: Pass/fail metrics do not capture partial correctness or code quality beyond correctness.

Hardware Specificity: Measurements on Apple M4 may not transfer to GPU clusters or other deployment environments. Relative trends likely generalize; absolute numbers may differ.

10 Future Work

- **Negative test cases:** Add invalid/malformed queries to measure specificity and false positive rates
- **Domain expansion:** General code generation (Python, JavaScript), text generation, reasoning tasks
- **Fine-tuning effects:** How task-specific fine-tuning affects scaling laws
- **MoE architectures:** Scaling laws for mixture-of-experts models
- **Confidence calibration:** Do models know when they’re wrong?
- **Temporal dynamics:** Stability of scaling laws across model versions

11 Conclusion

This work addresses a fundamental question: if LLMs are unreliable at mathematics, how can they still be useful for mathematical tasks? The answer lies in reframing their role—from *calculator* to *code generator*.

The core insight: LLMs cannot reliably compute $\int x^2 dx$, but they can reliably generate `Integrate[x^2, x]` for Wolfram Mathematica to execute. This “LLM as translator” paradigm achieves 94.4% accuracy by leveraging LLM strengths (language-to-code translation) while avoiding their weaknesses (mathematical computation).

I establish inference scaling laws as a complementary framework to training scaling laws. While Kaplan et al. (2020) and Hoffmann et al. (2022) optimize model creation, inference scaling laws optimize model selection for deployment.

Key findings:

1. **The paradigm works:** LLMs can reliably generate verified mathematical code (94.4% accuracy with qwen2.5:7b)
2. **Family-specific scaling:** Only 2 of 16 families exhibit strong power-law scaling ($R^2 > 0.98$)
3. **Task-specific behavior:** Mathematical code generation shows different patterns than general language modeling
4. **Energy-critical tradeoffs:** $17\times$ energy difference for statistically insignificant accuracy improvement (0.5%)
5. **Diagnostic metrics matter:** Precision, recall, and F1 reveal model “archetypes”—production-ready vs. overconfident vs. conservative
6. **Empirical validation required:** Universal scaling assumptions do not hold

Bottom line for practitioners: qwen2.5:7b achieves 94.4% precision, 100% recall ($F1 = 97.1\%$) at 2.9J/query—the optimal trade-off for production deployment. The pattern generalizes: for any domain requiring correctness guarantees, use LLMs to generate formal code and let verified systems execute it.

Broader implication: This work demonstrates a practical architecture for deploying LLMs in domains where correctness matters. Rather than trusting LLM outputs directly, we can use them as a translation layer between human intent and formal systems that compute reliably.

References

- Deng, Y., Yu, D., and Zhao, C. From Explicit CoT to Implicit CoT: Learning to Internalize CoT Step by Step. *arXiv preprint arXiv:2405.14838*, 2024.
- Kaplan, J., McCandlish, S., Henighan, T., et al. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.
- Hestness, J., Narang, S., Ardalani, N., et al. Deep Learning Scaling is Predictable, Empirically. *arXiv preprint arXiv:1712.00409*, 2017.
- Bahri, Y., Dyer, E., Kaplan, J., et al. Explaining neural scaling laws. *PNAS*, 121(27), 2024.
- OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023.